

Activity 9: Abstract Classes & Template Method Pattern

Objective

In this activity, you are going to improve an existing **Bank Account Management System** using:

- Abstract Classes
- Inheritance
- Method Overriding
- Template Method Pattern
- Exception Handling

You will have different types of bank accounts:

- Savings Account
- Current Account
- Salary Account
- Fixed Deposit Account

Although these accounts are different, they have some common information and common operations.

Your task is to identify what should be **common to all accounts** and what should be **different for each account type**.



Banking Scenario

Imagine that a bank has different types of accounts.

Every account has some common information:

- Account Number
- Customer Name
- Age
- Balance
- Account Type
- Account Status
- PIN

All accounts should support common operations such as:

- Deposit money
- Validate PIN
- Change PIN
- Display account information
- Withdraw money

However, the **withdrawal rule is different for each account type**.

For example:

Savings Account

A customer should not be allowed to withdraw money if the remaining balance goes below the minimum balance.

Current Account

A customer can withdraw more than the available balance, but only up to the allowed overdraft limit.

Salary Account

The customer cannot withdraw more than the available balance.

Fixed Deposit Account

A customer cannot withdraw money before the Fixed Deposit matures.

Part 1 – Create the Abstract Account

Create an abstract class:

```
AbstractAccount
```

This class should contain the information that is common to all account types.

Common fields

The class should maintain:

- accountNumber
- name
- age
- balance
- accountType
- status
- pin

Use appropriate access modifiers.

Constructor

Create a constructor that initializes all common account information.

Also apply these validations:

1. Customer age must be **18 or above**.
2. Initial balance cannot be negative.
3. PIN must contain exactly **4 digits**.

If any validation fails, throw an appropriate exception such as `IllegalArgumentException`.

Part 2 – Common Account Operations

Implement the following methods inside `AbstractAccount`.

1. Deposit

Create:

```
deposit(double amount)
```

Rules:

- Deposit amount must be greater than `0`.
- If the amount is invalid, throw `InvalidAmountException`.
- Otherwise, add the amount to the account balance.

Example

If the balance is:

```
Rs 5000
```

and the customer deposits:

```
Rs 2000
```

The new balance should be:

```
Rs 7000
```

2. Validate PIN

Create:

```
validatePin(String enteredPin)
```

The method should return:

- `true` → if the entered PIN is correct.

- `false` → if the entered PIN is incorrect.
-

3. Change PIN

Create:

```
changePin(String oldPin, String newPin)
```

The new PIN should be changed only when:

- The old PIN is correct.
- The new PIN contains exactly 4 digits.

Otherwise, return `false`.

4. Display Account Information

Create:

```
displayAccountInfo()
```

Display information such as:

```
Account Number: A1001
Name: Ravi
Age: 30
Balance: Rs 10000.0
Account Type: SAVINGS
Status: ACTIVE
```

Part 3 – Create the Template Method for Withdrawal

This is the most important part of today's activity.

Create the following method inside `AbstractAccount`:

```
withdraw(double amount, String enteredPin)
```

The withdrawal process must always follow the **same sequence**.

Step 1 – Validate PIN

First check whether the entered PIN is correct.

If the PIN is incorrect:

`InvalidPinException`

should be thrown.

Step 2 – Check Account Status

After validating the PIN, check the account status.

Only an account with status:

`ACTIVE`

can withdraw money.

If the account is inactive:

`InactiveAccountException`

should be thrown.

Step 3 – Validate Withdrawal Amount

The withdrawal amount must be greater than `0`.

If the amount is `0` or negative:

`InvalidAmountException`

should be thrown.

Step 4 – Perform the Actual Debit

After all the above validations pass, call:

`processDebit(amount)`

This method should **not be implemented in** `AbstractAccount`.

Instead, declare it as an abstract method:

```
processDebit(double amount)
```

Every child class will provide its own implementation.

💡 **Think about why this is useful:**

The withdrawal process is the same for every account, but the actual debit rule is different.

This is the main idea behind the **Template Method Pattern**.

Part 4 – Savings Account

The `SavingsAccount` class should extend:

```
AbstractAccount
```

It already has:

- `minBalance`
- `interestRate`

Implement `processDebit()`

Before deducting money, check:

```
remaining balance >= minimum balance
```

If withdrawing the requested amount would make the balance lower than the minimum balance:

Throw:

```
MinimumBalanceViolationException
```

Otherwise, deduct the amount.

Example

Current balance:

```
Rs 10000
```

Minimum balance:

Rs 1000

Withdrawal:

Rs 2000

Result:

Withdrawal successful
Balance: Rs 8000

But if:

Current balance:

Rs 10000

Minimum balance:

Rs 1000

Withdrawal:

Rs 9500

Then the remaining balance would be:

Rs 500

This is below the minimum balance.

Therefore:

MinimumBalanceViolationException

should be thrown.

Part 5 – Current Account

The `CurrentAccount` class should extend:

`AbstractAccount`

It has an:

`overdraftLimit`

Implement `processDebit()`

A Current Account is allowed to go below zero, but only up to its overdraft limit.

You need to check whether:

`withdrawal amount <= balance + overdraft limit`

If the customer tries to withdraw more than the available balance plus overdraft limit:

Throw:

`InsufficientBalanceException`

Otherwise, deduct the amount.

Example

Balance:

`Rs 2000`

Overdraft limit:

`Rs 5000`

Maximum amount customer can use:

`Rs 7000`

Therefore:

Withdrawal of `Rs 5000` →  Successful

New balance:

`Rs -3000`

But withdrawal of `Rs 8000` →  Not allowed

because it exceeds the overdraft limit.

Part 6 – Salary Account

The `SalaryAccount` class should extend:

`AbstractAccount`

It contains:

- `employerName`
- `inactiveMonths`

Implement `processDebit()`

A Salary Account should not allow the customer to withdraw more than the available balance.

If:

`withdrawal amount > balance`

throw:

`InsufficientBalanceException`

Otherwise, deduct the amount.

Example

Balance:

`Rs 15000`

Withdrawal:

`Rs 5000`

Result:

`Balance: Rs 10000`

But:

Balance:

`Rs 15000`

Withdrawal:

Rs 20000

Result:

InsufficientBalanceException

Part 7 – Fixed Deposit Account

The `FixedDepositAccount` class should extend:

`AbstractAccount`

It contains:

- `tenureMonths`
- `interestRate`

Implement `processDebit()`

A Fixed Deposit does **not allow premature withdrawal**.

Therefore, whenever `processDebit()` is called, throw:

`AccountException`

with a suitable message such as:

Premature withdrawal not allowed on Fixed Deposit

💡 Notice something important:

You don't need to check the balance here.

The business rule is simply:

Fixed Deposit → No premature withdrawal.

Think Before You Code

Before starting, answer these questions:

Question 1

Which fields are common to all account types?

👉 These belong in `AbstractAccount`.

Question 2

Which operation follows the same sequence for every account?

👉 Think about `withdraw()`.

Question 3

Which part of withdrawal changes depending on the account type?

👉 Think about `processDebit()`.

Question 4

Why should `processDebit()` be abstract?

👉 Because each account has a different debit rule.

Question 5

Where should the PIN validation happen?

👉 Before calling `processDebit()`.

Question 6

Where should account status be checked?

👉 Before processing the debit.

Test Your Implementation

After completing the classes, create/run:

```
TestAbstractAccount
```

Your test program should verify at least the following scenarios.

Test 1 – Successful Savings Withdrawal

```
Savings Balance = Rs 10000  
Withdrawal = Rs 2000  
PIN = Correct  
Status = ACTIVE
```

Expected:

```
[Savings] Withdraw 2000: SUCCESS | Balance: Rs 8000.0
```

Test 2 – Savings Minimum Balance

Try to withdraw an amount that causes the balance to fall below the minimum balance.

Expected:

```
[Savings] Withdraw below min balance:  
Caught MinimumBalanceViolationException [PASS]
```

Test 3 – Current Account Overdraft

Example:

```
Balance = Rs 2000  
Overdraft Limit = Rs 5000  
Withdrawal = Rs 5000
```

Expected:

```
[Current] Overdraft debit: SUCCESS | Balance: Rs -3000.0
```

Test 4 – Fixed Deposit Withdrawal

Try withdrawing money from a Fixed Deposit before maturity.

Expected:

```
[FixedDeposit] Premature debit:  
Caught AccountException [PASS]
```

★ Expected Final Output

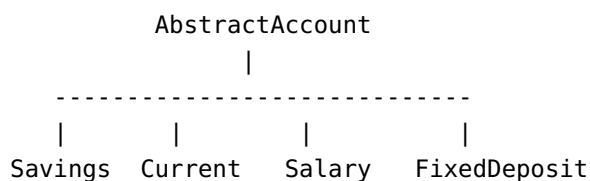
If your implementation is correct, you should see something similar to:

```
=== Activity 9: Abstract Account & Template Pattern ===  
  
[Savings] Withdraw 2000: SUCCESS | Balance: Rs 8000.0  
  
[Savings] Withdraw below min balance:  
Caught MinimumBalanceViolationException [PASS]  
  
[Current] Overdraft debit: SUCCESS | Balance: Rs -3000.0  
  
[FixedDeposit] Premature debit:  
Caught AccountException [PASS]  
  
Template method pattern executed successfully!
```

🚀 Your Challenge

Do **not** copy the implementation directly.

Try to design the solution yourself using the following structure:



Remember:

Common behavior → **AbstractAccount**

Different debit behavior → **processDebit()** in child classes

Common withdrawal sequence → **withdraw()** in **AbstractAccount**

Learning Outcome

By completing this activity, you should be able to explain:

“The Template Method Pattern defines the overall workflow in the parent class, while subclasses provide their own implementation for the steps that vary.”

Once you finish coding, make sure all test cases pass before submitting your work.